



RUB

RUHR-UNIVERSITÄT BOCHUM

OUT OF ORDER EXECUTION

Computer Architecture

Agenda

- Register renaming
- Data flow analysis
- Tomasulo's Algorithm
- Speculative Execution
- Exceptions

Pipeline Diagram Types

- We have seen stage-centric diagrams
- Instruction-centric diagrams are an alternative

	1	2	3	4	5	6	7	8
addi t1, t1, 0	FE	DE	EX	M	WB			
div t2, t0, t3		FE	DE	EX	M	WB		
mul t0, t1, t0			FE	--	--	DE	EX	M
addi t1, t3, t2				FE	--	--	DE	EX

Instruction centric

I_1 : addi t1, t1, 0
 I_2 : div t2, t0, t3
 I_3 : mul t0, t1, t0
 I_4 : addi t1, t3, t2

	FE	DE	EX	M	WB
1	I_1				
2	I_2	I_1			
3	I_3	I_2	I_1		
4	I_4		I_2	I_1	
5				I_2	I_1
6		I_3			I_2
7		I_4	I_3		
8			I_4	I_3	

Stage centric

Forwarding

I_1 : addi t1, t1, 0
 I_2 : div t2, t0, t3
 I_3 : mul t0, t1, t0
 I_4 : addi t1, t3, t2

Without forwarding

	1	2	3	4	5	6	7	8	9	10
addi t1, t1, 0	FE	DE	EX	M	WB					
div t2, t0, t3		FE	DE	EX	M	WB				
mul t0, t1, t0			FE	--	--	DE	EX	M	WB	
addi t1, t3, t2				FE	--	--	DE	EX	M	WB

With forwarding (from M)

addi t1, t1, 0	FE	DE	EX	M	WB					
div t2, t0, t3		FE	DE	EX	M	WB				
mul t0, t1, t0			FE	--	DE	EX	M	WB		
addi t1, t3, t2				FE	--	DE	EX	M	WB	

With forwarding (from EX)

addi t1, t1, 0	FE	DE	EX	M	WB					
div t2, t0, t3		FE	DE	EX	M	WB				
mul t0, t1, t0			FE	DE	EX	M	WB			
addi t1, t3, t2				FE	DE	EX	M	WB		

What if div is slow?

I_1 : `addi t1, t1, 0`
 I_2 : `div t2, t0, t3`
 I_3 : `mul t0, t1, t0`
 I_4 : `addi t1, t3, t2`

Div takes 3 cycles

	1	2	3	4	5	6	7	8	9	10
<code>addi t1, t1, 0</code>	FE	DE	EX	M	WB					
<code>div t2, t0, t3</code>		FE	DE	EX	EX	EX	M	WB		
<code>mul t0, t1, t0</code>			FE	DE	--	--	EX	M	WB	
<code>addi t1, t3, t2</code>				FE	DE	--	--	EX	M	WB

Recap: Scoreboarding

Multiple Functional Units

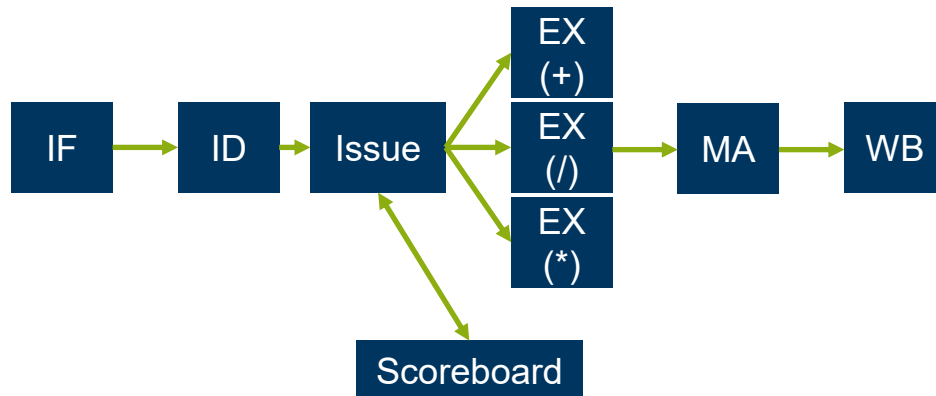
Scoreboard to keep track of dependencies

Problem:

With pipelining we only need to wait for true dependencies

Here, we need to wait for all dependencies

Can we go more “Out of Order”?



Scoreboarding

I_1 : addi t1, t1, 0
 I_2 : div t2, t0, t3
 I_3 : mul t0, t1, t0
 I_4 : addi t1, t3, t2

Div takes 3 cycles

	1	2	3	4	5	6	7	8	9	10
addi t1, t1, 0	FE	DE	EX	M	WB					
div t2, t0, t3		FE	DE	EX	EX	EX	M	WB		
mul t0, t1, t0			FE	DE	--	--	EX	M	WB	
addi t1, t3, t2				FE	DE	--	--	EX	M	WB

Scorboarding

	1	2	3	4	5	6	7	8	9	10
addi t1, t1, 0	FE	DE	EX	M	WB					
div t2, t0, t3		FE	DE	EX _D	EX _D	EX _D	M	WB		
mul t0, t1, t0			FE	DE	EX _M	M	--	--	WB	
addi t1, t3, t2				FE	--	--	DE	EX _A	M	WB

What Does That Mean?

Scoreboarding:

Any dependency enforces order of execution

What we want:

A mechanism that allows executing in any order irrespective of dependencies

Re-Ordering: Write after Write

I_n : ADDI t0, t1, 1

I_m : MUL t0, t1, t1



I_m : MUL t0, t1, t1

I_n : ADDI t0, t1, 1

Problem:

Value from I_n is in t0 after all writes are complete

Future instructions read the wrong value

Re-Ordering: Write after Write

Solution:

Let I_n write into "virtual" register

All other Instructions that read the value from I_n read from that register

 **Register renaming**

I_n : ADDI t0, t1, 1

I_m : MUL t0, t1, t1



I_m : MUL t0, t1, t1

I_n : ADDI v0, t1, 1

Re-ordering: Write After Read

I_n : ADDI t1, t0, 1

I_m : MUL t0, t2, t3



I_m : MUL v0, t2, t3

I_n : ADDI t1, t0, 1

Does that also work for Anti Dependencies?

Yes:

Use a different register for the result of I_m

I_n can still read the original value of t0

Instructions that use the value from I_m use the other register

Re-ordering: Read After Write

I_n : ADDI t0, t1, 1

I_m : MUL t2, t0, t3



I_m : MUL t2, t0, t3

I_n : ADDI t0, t1, 1

Does that also work for True Dependencies?

No!

I_m uses the value computed by I_n

Determining the Instruction Flow

So far: What can we execute in parallel?

Now: How can we determine order of execution

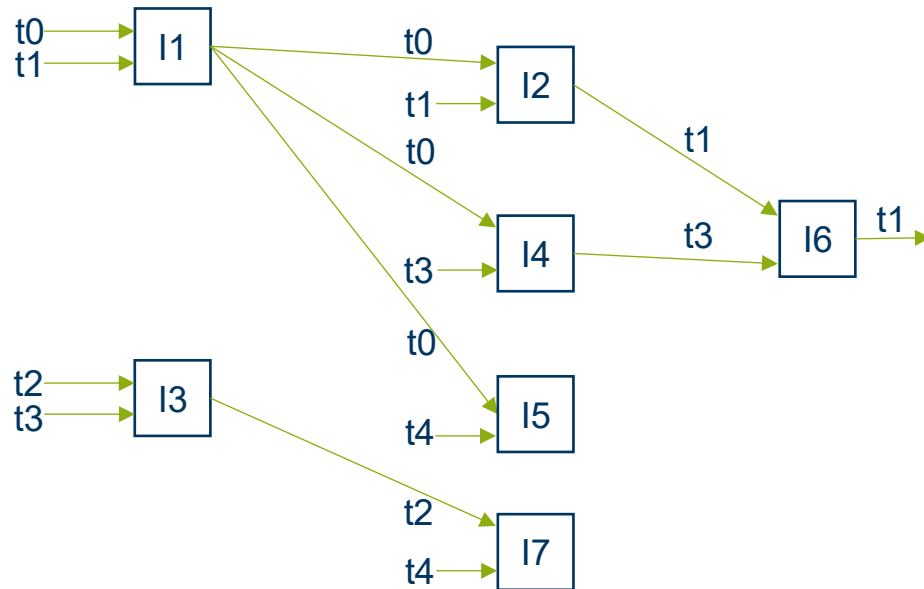
➡ Data Dependency Graph:

- Nodes represent instructions
- Arrows indicate that the output of an instruction is used by another
- Arrows pointing towards a node are its inputs
- Arrows coming from a node are the results



Data Dependency Graph: Example

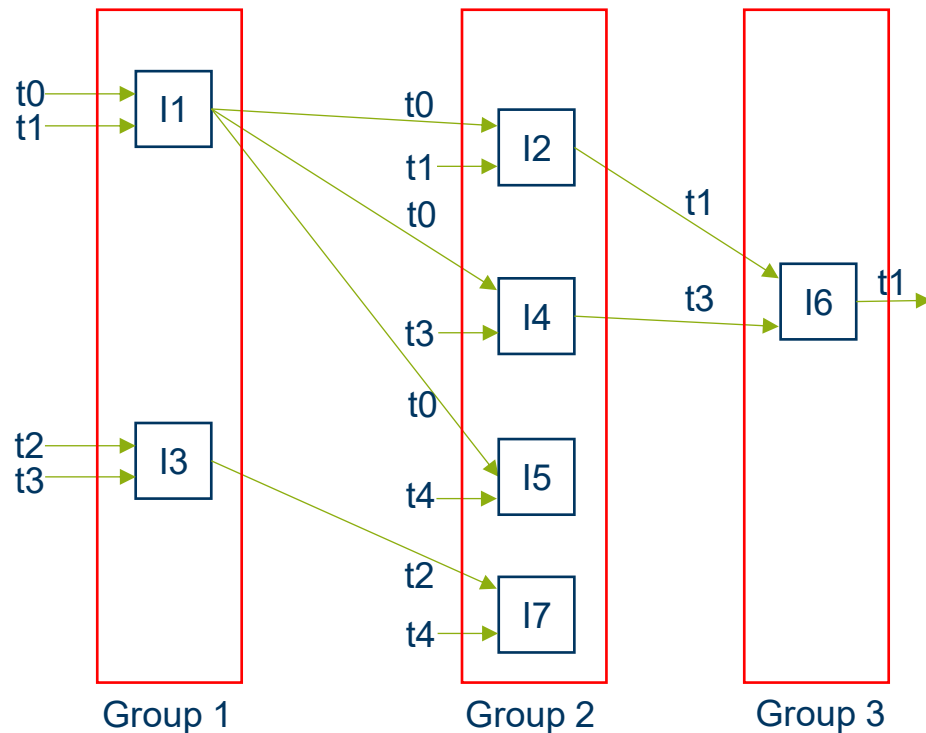
Instruction	Writes	Reads
I1	t0	t0, t1
I2	t1	t0, t1
I3	t2	t2, t3
I4	t3	t0, t3
I5	t0	t0, t4
I6	t1	t1, t3
I7	t2	t2, t4



Data Dependency Graph

Now we know:

- Which instructions can be executed simultaneously
- Which instructions have to wait



Data Flow Analysis

Use data flow model to decide the optimal sequence of instructions and carry them out in that order

➡ We just did that

Advantages:

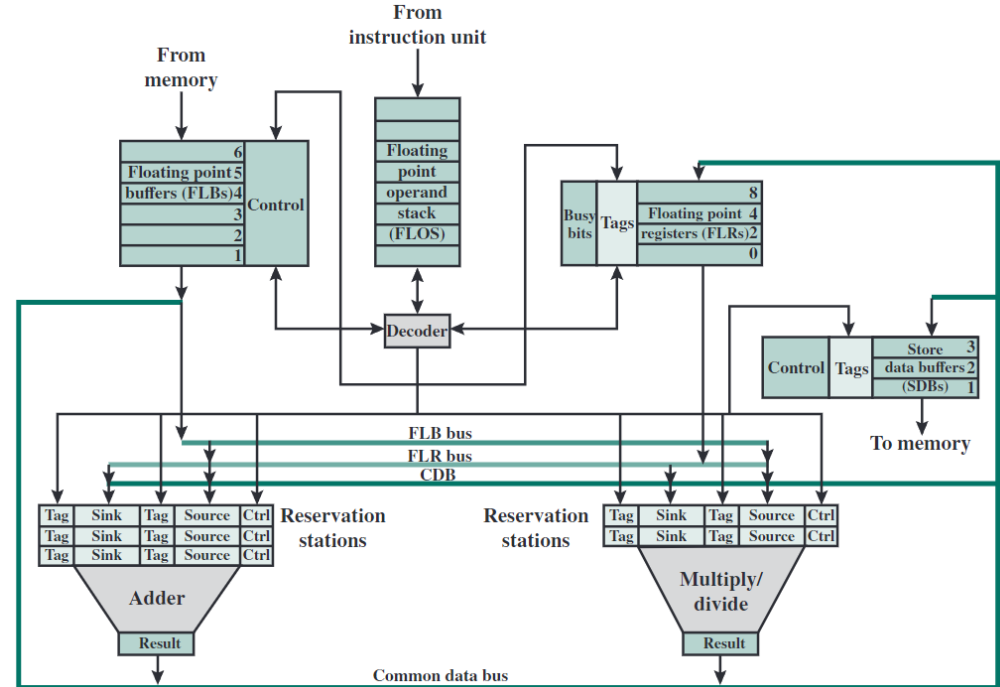
- **Improved control flow:** instructions are carried out when they are ready and not at the time they are specified in the program
- **Prevent unnecessary delay:** Keep processor as busy as possible

Tomasulo's Algorithm

Scheduling Algorithm
similar to Scoreboarding
that uses **register
renaming** and **data flow
analysis**

Additionally:

Reservation stations: FU's
waiting room for n
instructions



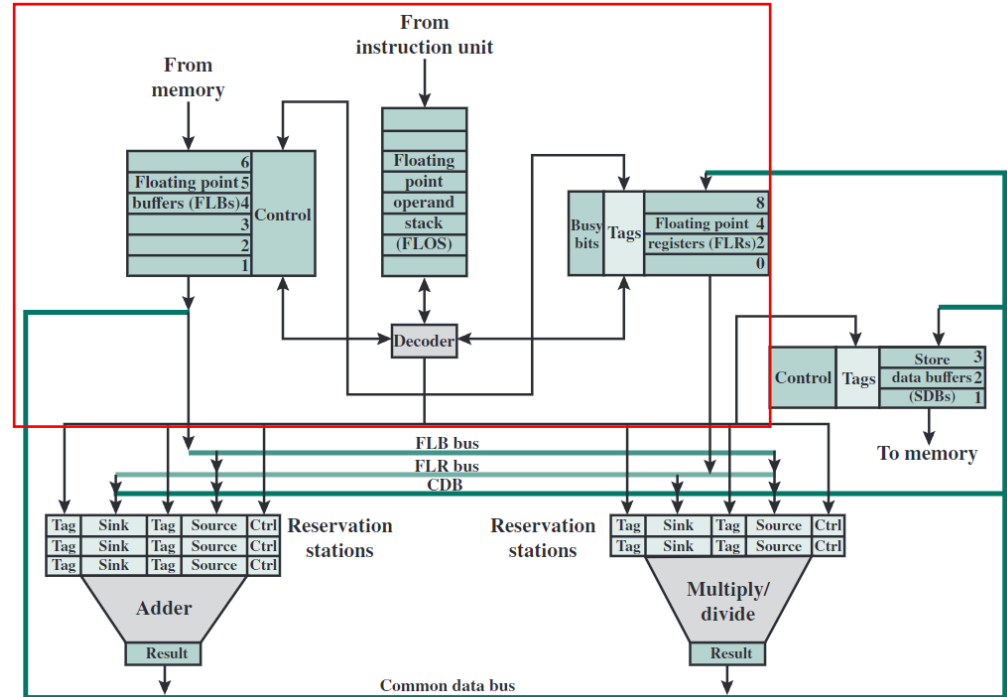
"Basic Structure of a Floating-Point Unit Using Tomasulo's Algorithm" from "Computer Organization and Architecture" by William Stallings 11. Edition, page 841

Stages: Fetch

Fetch Instructions from memory

Instructions are fetched in batches of m instructions at a time

To accommodate re-ordering of dependent instructions registers are renamed



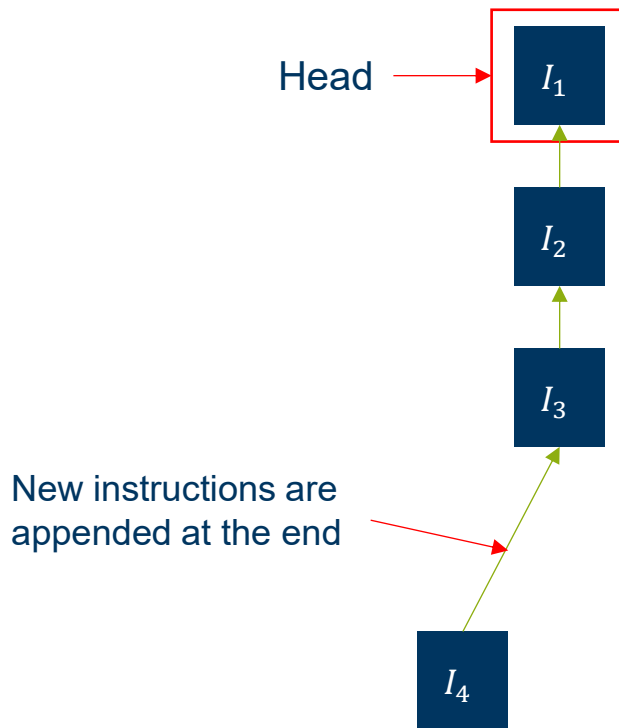
"Basic Structure of a Floating-Point Unit Using Tomasulo's Algorithm" from "Computer Organization and Architecture" by William Stallings 11. Edition, page 841

Re-Order Buffer

Fetches instructions are also put into a buffer ascending by their index

Instructions are committed in-order as only the head can be committed

When the head is committed the instruction after it becomes the new head



Register Modification

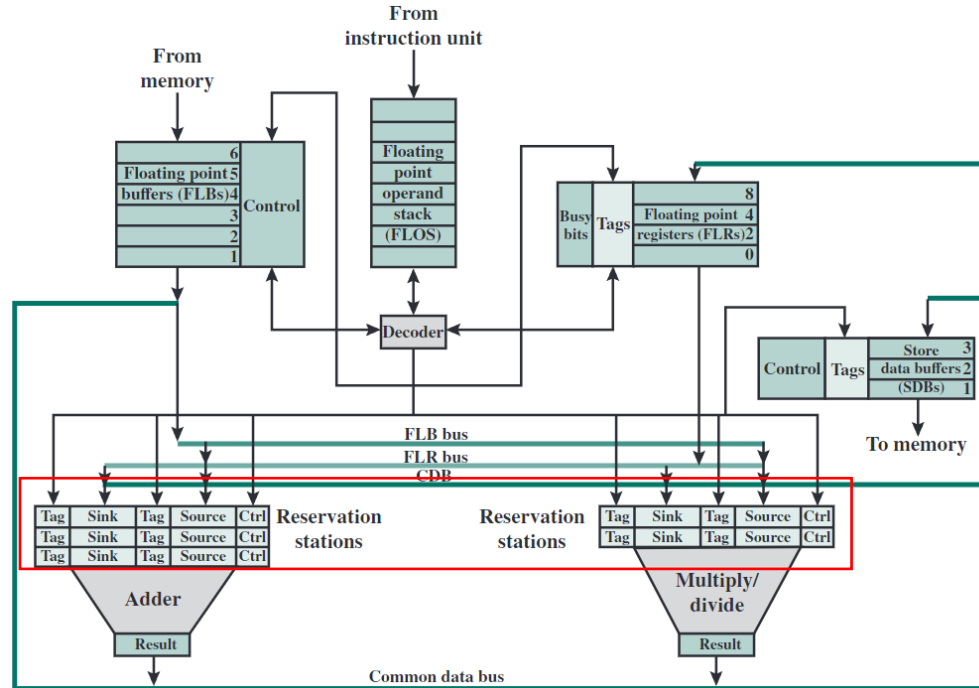
Add Columns "Busy" and "Producer"

Busy: There is an instruction in a functional unit about to write into this register

Producer: Identifier of that instruction

Name	Value	Busy	Producer
t0	0x102A384B	1	I1
t1	0xAFFFFFFF	0	--
t2	0x00000000	0	--
t3	0xEB45A368	1	I3
t4	0xD123E234	1	I4

Stages: Issue



"Basic Structure of a Floating-Point Unit Using Tomasulo's Algorithm" from "Computer Organization and Architecture" by William Stallings 11. Edition, page 841

Stages: Issue

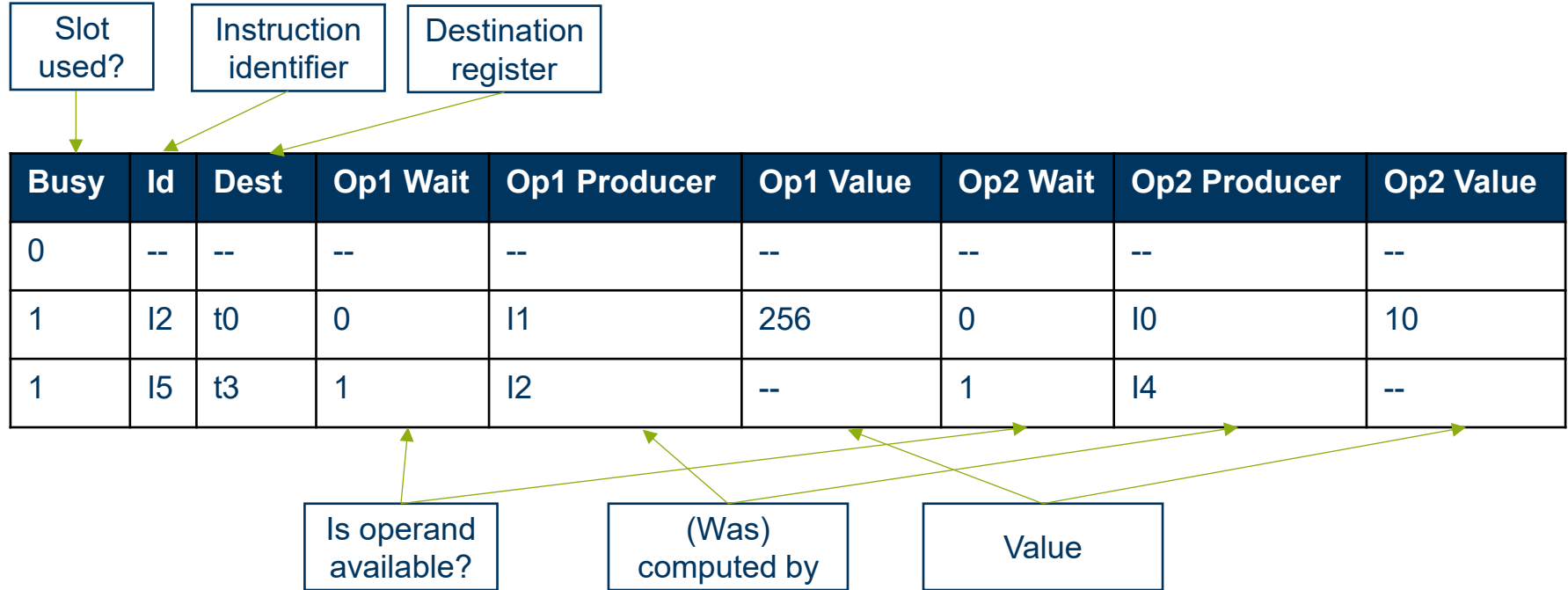
Choose the reservation station the instruction will be issued to based on the type of the instruction and reservation station availability

Instruction stays in the reservation station until all operands are available

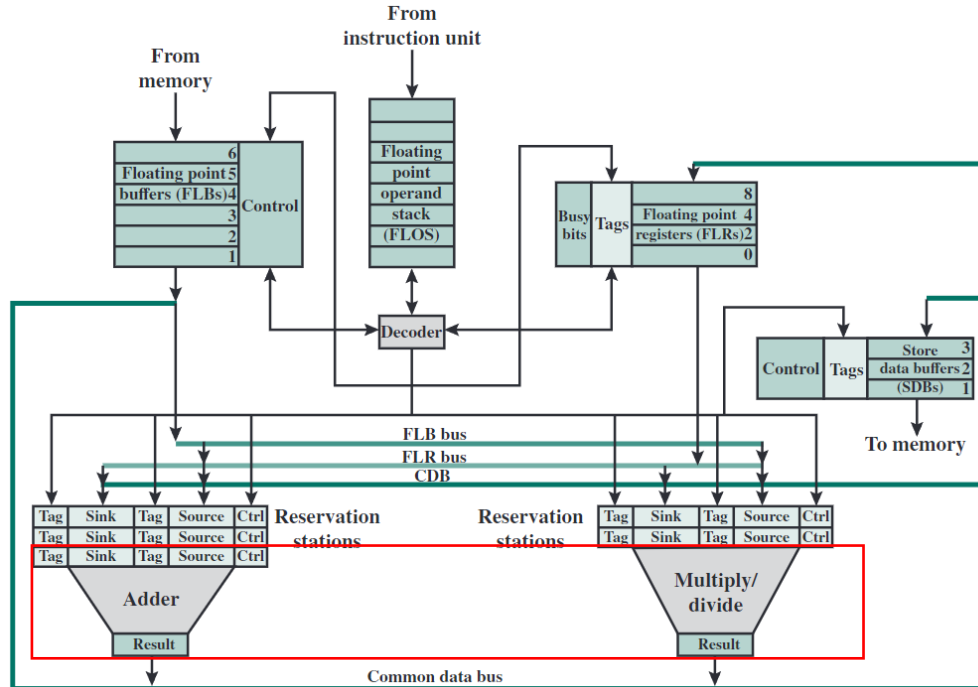
Then it can be pushed into the FU and be executed

Anti and Output dependencies are eliminated by using virtual registers and renaming all references in other instructions

Reservation Stations



Stages: Execution



"Basic Structure of a Floating-Point Unit Using Tomasulo's Algorithm" from "Computer Organization and Architecture" by William Stallings 11. Edition, page 841

Stages: Execution

Instructions from the reservation station are executed as soon as all operands have been computed

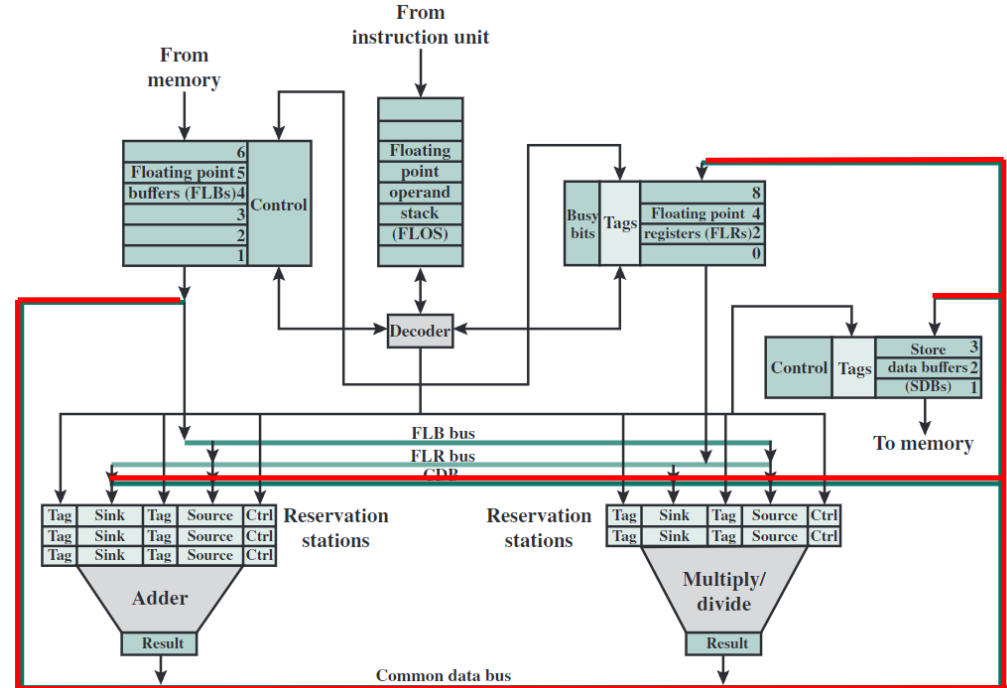
Busy	Id	Dest	Op1 Wait	Op1 Producer	Op1 Value	Op2 Wait	Op2 Producer	Op2 Value
0								
1	I2	t0	0	I1	256	0	I0	10
1	I5	t3	1	I2		1	I4	

I2 is the next instruction to be executed as both waiting flags are set to 0 while I5 still has to wait for the results from Instructions I2 and I4

Stages: Write Back

After execution, the result of an instructions is put on the common data bus

The reservation stations monitor the common data bus and update their table entries if possible



"Basic Structure of a Floating-Point Unit Using Tomasulo's Algorithm" from "Computer Organization and Architecture" by William Stallings 11. Edition, page 841

Stages: Commit

If the instruction is in the head position of the re-order buffer it is committed immediately

If not, it is committed as soon as it reaches the head position



Speculative Execution

Recap: Branch Prediction

Decide if it is more likely that the branch is taken or not

Prefetch these instructions and execute them

If enough resources are available:

Prefetch multiple branches ahead

If success quote is high:

Very efficient solution that makes use of unneeded resources

Speculative Execution

Branch prediction



Execute one or more
branches ahead, keeping
results in virtual locations

+

Data flow analysis



Decide optimal sequence for execution



Processor is as busy as possible with executing instructions that are likely to be needed

Eager and Lazy Execution

Alternatives to branch prediction:

Eager execution:

Execute as many instructions forward as possible

Always evaluate both sides of the branch

 Resource need grows exponentially with every branch but also efficiency

Lazy execution:

No execution forward

Execute only the side of the branch that has been taken

 Inefficient but saves resources

Exceptions

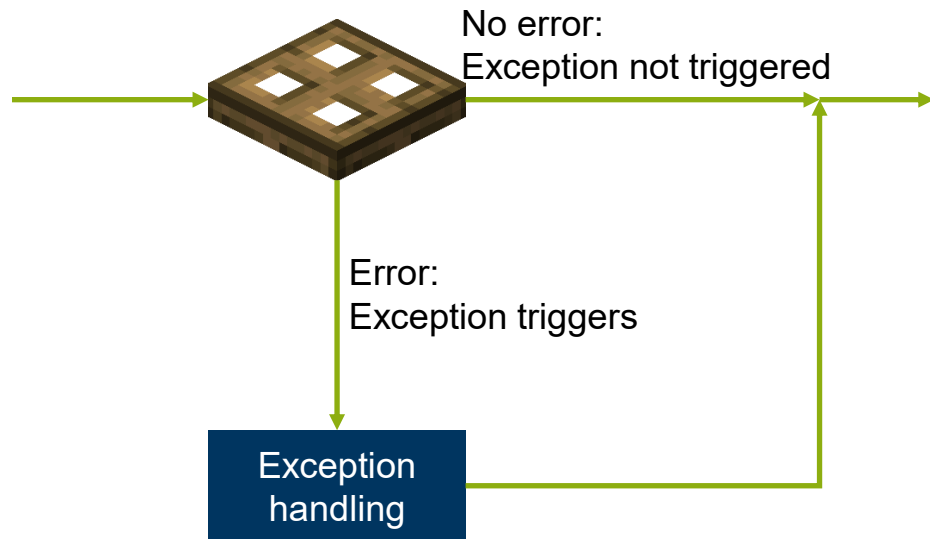
Execution errors

- Some instructions can have errors
 - Divide by zero
 - Access to a bad memory location
 - Illegal instruction
- Possible ways of handling
 - Ignore
 - Fix and continue
 - Kill program
- The processor cannot decide what to do

Exceptions

Used for Error handling

- If error occurs exception is thrown
- Processor gives control to the operating system, which handles the exception
- Typically
 - Save program state
 - Change program counter to exception handler



Exception handling

Pre-written responses to possible errors

If program cannot handle an exception, it is passed on to the next-highest program

An exception cannot be ignored and has to be caught and handled eventually

And now it's your turn...

When executing a program using Tomasulo's Algorithm and an error occurs, at what stage of execution is the exception thrown?